LeetCode 841: Keys and Rooms

1. Problem Title & Link

Title: LeetCode 841: Keys and Rooms

Link: https://leetcode.com/problems/keys-and-rooms/

2. Problem Statement (Short Summary)

There are n rooms numbered:

0 to n-1

Initially:

Room 0 is unlocked

Each room contains keys to other rooms.

You can enter only unlocked rooms.

Return:

True

if all rooms can be visited.

Otherwise:

False

3. Examples (Input → Output)

Example 1

Input:

rooms = [

[1],

[2],

[3],

[]

]

Output:

True

Explanation:

Room 0 → Key 1

Room 1 → Key 2

Room 2 → Key 3

All rooms visited.

Example 2

Input:

rooms = [

[1,3],

[3,0,1],

[2],

[0]

]

Output:

False

Room 2 can never be reached.

4. Constraints

- n == rooms.length
- 2 ≤ n ≤ 1000
- 0 ≤ rooms[i].length ≤ 1000

5. Core Concept (Pattern / Topic)

Graph Traversal (DFS / BFS)

Related Topics:

- Graphs
- Depth First Search
- Breadth First Search
- Reachability

6. Thought Process (Step-by-Step Explanation)

Convert Problem Into Graph

Room:

0

contains:

[1,3]

means:

0 → 1

0 → 3

This is exactly a graph.

Example:

rooms = [
[1],
[2],
[3],
[]
]
Graph:
0 → 1 → 2 → 3
Question becomes:
Starting from node 0,
can we reach every node?

Brute Force
For every room:
Try exploring all possibilities.
Repeated work.
Not efficient.

Better Idea
Use:
DFS
or
BFS
Starting from:
room 0
Collect all reachable rooms.

Key Observation
Every time we enter a room:
We get new keys
which allow us to visit more rooms.
This is exactly graph traversal.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
rooms = [
 [1],
 [2],
```

```
 [3],
 []
]
Graph:
0
|
v
1
|
v
2
|
v
3
DFS visits:
0 → 1 → 2 → 3
All rooms visited.
Answer:
True
```

8. Pseudocode

```
visited = set()

DFS(room):

    mark room visited

    for every key:

        if not visited:

            DFS(key)

Start DFS(0)

return visited_count == n
```

9. Code Implementation

✅ **Python (DFS)**

```
class Solution:

    def canVisitAllRooms(self, rooms):

        visited = set()

        def dfs(room):
```

```
            visited.add(room)
            for key in rooms[room]:
                if key not in visited:
                    dfs(key)
        dfs(0)
```

return len(visited) == len(rooms)

✅ **Python (BFS)**

```
from collections import deque

class Solution:
    def canVisitAllRooms(self, rooms):
        visited = {0}
        queue = deque([0])
        while queue:
            room = queue.popleft()
            for key in rooms[room]:
                if key not in visited:
                    visited.add(key)
                    queue.append(key)
        return len(visited) == len(rooms)
```

✅ **Java (DFS)**

```
class Solution {

    public boolean canVisitAllRooms(
            List<List<Integer>> rooms) {

        boolean[] visited =
                new boolean[rooms.size()];
```

```
        dfs(0, rooms, visited);

        for (boolean room : visited) {

            if (!room)
                return false;
        }

        return true;
    }

    private void dfs(
            int room,
            List<List<Integer>> rooms,
            boolean[] visited) {

        visited[room] = true;

        for (int key : rooms.get(room)) {

            if (!visited[key]) {

                dfs(
                    key,
                    rooms,
                    visited
                );
            }
        }
    }
}
```

✅ **Java (BFS)**

```
import java.util.*;

class Solution {

    public boolean canVisitAllRooms(
            List<List<Integer>> rooms) {

        boolean[] visited =
                new boolean[rooms.size()];

        Queue<Integer> queue =
                new LinkedList<>();

        queue.offer(0);
```

```
        visited[0] = true;

        while (!queue.isEmpty()) {

            int room = queue.poll();

            for (int key : rooms.get(room)) {

                if (!visited[key]) {

                    visited[key] = true;
                    queue.offer(key);
                }
            }
        }

        for (boolean room : visited) {

            if (!room)
                return false;
        }

        return true;
    }
}
```

10. Time & Space Complexity

Time Complexity

Each room visited once.

Each key processed once.

O(V + E)

where:

V = rooms

E = keys

Space Complexity

O(V)

Visited array + recursion/queue.

11. Common Mistakes / Edge Cases

Mistake 1

Not marking room visited before DFS.

Can cause cycles.

Mistake 2

Revisiting rooms repeatedly.

Leads to unnecessary work.

Mistake 3

Checking keys instead of reachable rooms.

Need graph traversal.

12. Detailed Dry Run

Input:

rooms = [

[1],

[2],

[3],

[]

]

Visited:

{}

Start DFS(0)

Visited:

{0}

Keys:

[1]

Go to:

1

DFS(1)

Visited:

{0,1}

Keys:

[2]

Go to:

2

DFS(2)

Visited:

{0,1,2}

Keys:

[3]

Go to:

3

DFS(3)

Visited:

{0,1,2,3}

No keys.

All rooms visited.

Answer:

True

13. Common Use Cases

Building Access Control

Rooms and keys.

Network Reachability

Can all nodes be reached?

Social Network Traversal

Visit connected users.

Dependency Resolution

Follow references/dependencies.

14. Common Traps

- Forgetting visited array
- Infinite recursion on cycles
- Starting DFS from every room
- Treating it as a matrix problem

15. Builds To (Related Problems)

1. LC 200 — Number of Islands
2. LC 133 — Clone Graph
3. LC 547 — Number of Provinces
4. LC 841 — Keys and Rooms
5. LC 797 — All Paths From Source to Target

16. Alternate Approaches

Approach	Time	Space	Notes
DFS	O(V+E)	O(V)	Optimal
BFS	O(V+E)	O(V)	Optimal

17. Why This Solution Works

Each room acts as a graph node and each key acts as a directed edge. Starting from room 0, DFS/BFS explores every reachable room. If the number of visited rooms equals the total number of rooms, then all rooms can be visited.

18. Variations / Follow-Up Questions

Follow-Up 1

Count connected components.

→ LC 547 (Number of Provinces)

Follow-Up 2

Find shortest path to a room.

→ BFS

Follow-Up 3

Return visiting order.

Store traversal sequence during DFS/BFS.